

C语言指针问题解答

1. 什么是指针

指针是 C 语言提供了一种特殊变量类型，用于存储**内存地址**。
通过指针可以间接访问、修改它所指向的内存中的数据。

2. 如何在 C 语言中定义指针变量

语法：

```
类型 *指针变量名;
```

例如：

```
int *p;        // 指向 int 的指针
char *q;       // 指向 char 的指针
double *r;     // 指向 double 的指针
```

也可以在定义时初始化：

```
int x = 10;
int *p = &x;
```

如果要一次定义多个指针，每个变量前都要加 *：

```
int *p, *q;    // p 和 q 都是指针
int *p, q;     // p 是指针，q 是 int 变量
```

3. 指针变量的大小是固定的吗？与什么有关

指针变量的大小**不是固定不变的**，它取决于：

- 目标平台架构，如 32 位系统或 64 位系统
- 编译器和编译目标
- 地址总线的宽度

常见情况：

平台	指针大小
32 位系统	通常 4 字节
64 位系统	通常 8 字节

在同一个平台上，不同类型的对象指针，如 `int *`、`char *`、`double *`，大小通常相同。但 C 标准并没有规定指针一定是 4 字节或 8 字节，只要求 `void *` 能容纳对象指针。

4. 代码输出结果及解释

输出结果

```
20 10
```

原因解释

第一步：修改 x

```
int x = 10;  
int* p = &x;  
*p = 20;
```

`p` 指向 `x`，`*p = 20` 把 `x` 的值改成 20。
所以最终 `x` 为 20。

第二步：分析 `arr` 和 `q`

```
int arr[3] = {3, 6, 9};  
int *q = arr;
```

- `arr` 是数组名，在表达式中通常退化为指向首元素的指针，即 `&arr[0]`
- `q = arr` 后，`q` 指向 `arr[0]`
- 此时：
 - `arr[0] = 3`
 - `arr[1] = 6`
 - `arr[2] = 9`
 - `q` 指向 `arr[0]`

第三步：计算 `++*arr`

`++*arr` 由于 `++` 和 `*` 从右向左结合，等价于：

```
++(*arr)
```

即：

1. `*arr` 是 `arr[0]`，值为 3
2. `++(*arr)` 把 `arr[0]` 自增为 4
3. 整个表达式的值为自增后的值，即 4

所以：

```
arr[0] 变为 4  
++*arr 的值为 4
```

第四步：计算 `*++q`

`*++q` 中 `++` 优先级高于 `*`，等价于：

```
*(++q)
```

即：

1. `q` 原来指向 `arr[0]`
2. `++q` 让 `q` 指向 `arr[1]`
3. `*++q` 取出 `arr[1]` 的值
4. `arr[1]` 原来是 6，没有被修改
5. 所以 `*++q` 的值为 6

第五步：计算 `y`

```
int y = ++*arr + *++q;
```

即：

```
y = 4 + 6;  
y = 10;
```

最终：

- `x = 20`
- `y = 10`

所以输出：

```
20 10
```

5. 什么是野指针

野指针是指指向**不确定、无效或已经不可用内存**的指针。

常见来源：

1. 未初始化的指针

```
int *p;  
*p = 10;    // 危险，p 指向未知地址
```

2. 指向已释放内存的指针

```
int *p = malloc(sizeof(int));  
free(p);  
*p = 10;    // 危险，p 已成为野指针
```

3. 指向超出作用域的局部变量

```
int *f() {  
    int x = 10;  
    return &x;    // 危险，x 在函数结束后失效  
}
```

4. 越界访问后的指针

```
int arr[3];  
int *p = arr + 5;    // 越界，危险
```

6. 野指针的危害

野指针会导致**未定义行为**，常见后果包括：

- 程序崩溃，如段错误 Segmentation Fault
- 数据被意外修改
- 内存泄漏或重复释放

- 程序行为不可预测
- 安全漏洞，如被利用进行攻击

因为野指针指向的地址可能无效，也可能恰好指向其他重要数据，所以危害很大。

7. 如何避免产生野指针

建议做到以下几点：

1. 指针定义时初始化

```
int *p = NULL;
```

2. 释放内存后立即置空

```
free(p);  
p = NULL;
```

3. 使用前检查指针是否有效

```
if (p != NULL) {  
    *p = 10;  
}
```

4. 不要返回局部变量的地址

```
int *f() {  
    static int x = 10;  
    return &x;    // 可以，static 变量生命周期长  
}
```

5. 注意数组边界，避免越界

```
for (int i = 0; i < 3; i++) {  
    arr[i] = 0;  
}
```

6. 动态分配内存后检查是否成功

```
int *p = malloc(sizeof(int));  
if (p == NULL) {
```

```
// 处理分配失败  
}
```

```
#include "stdio.h" void swap(int *a,int *b){ int temp; temp=*a; *a=*b; *b=temp; } int main(){ int a,b; scanf("%d %d",&a,&b); swap(&a,&b); printf("%d %d",a,b); return 0; }
```

C语言结构体问题解答

1. 什么是结构体

结构体是用户自定义的复合数据类型，可以把不同类型的数据组合在一起，形成一个新的数据类型。结构体中的每个数据称为“成员”，成员可以是基本类型、数组、指针，甚至其他结构体。

2. 完成 `PerInfo` 结构体定义，并使用 `typedef` 取别名

要求成员：

1. 个人姓名：字符型数组，长度为 10 个字节
2. 性别：字符型
3. 年龄：整型
4. 身高：双精度浮点型

最终结构体定义如下：

```
typedef struct PerInfo {  
    char name[10];    // 姓名，字符数组，长度 10  
    char gender;      // 性别，字符  
    int age;          // 年龄，整型  
    double height;    // 身高，双精度浮点型  
} PerInfo;
```

说明：

- `struct PerInfo` 是结构体标签名。
- `typedef struct PerInfo { ... } PerInfo;` 把 `struct PerInfo` 取别名为 `PerInfo`。
- 之后可以直接使用 `PerInfo` 定义变量，而不需要写成 `struct PerInfo`。

示例：

```
PerInfo p1;  
struct PerInfo p2;    // 也可以，因为标签名也叫 PerInfo
```

3. 结构体指针的含义

结构体指针是指向结构体变量的指针。

它存储的是结构体变量的起始地址。

定义方式：

```
PerInfo p;  
PerInfo *ptr = &p;
```

访问成员有两种方式：

```
(*ptr).name  
ptr->name
```

其中 `->` 是专门用于结构体指针访问成员的运算符。

示例：

```
#include <stdio.h>  
  
typedef struct PerInfo {  
    char name[10];  
    char gender;  
    int age;  
    double height;  
} PerInfo;  
  
int main() {  
    PerInfo p = {"Tom", 'M', 20, 175.5};  
    PerInfo *ptr = &p;  
  
    printf("%s\n", ptr->name);  
    printf("%c\n", ptr->gender);  
    printf("%d\n", ptr->age);  
    printf("%.1f\n", ptr->height);  
  
    return 0;  
}
```

结构体指针常用于函数传参、动态内存分配和链表等数据结构中。

4. 结构体内存对齐规则与大小计算

4.1 内存对齐规则

在常见的 64 位系统中，默认对齐规则通常如下：

- 1. 每个成员的起始偏移必须是该成员自身大小的整数倍。
- 2. 结构体的总大小必须是结构体中最大对齐成员的整数倍。
- 3. 编译器可能会在成员之间插入填充字节，以满足对齐要求。
- 4. 具体对齐方式与平台、编译器有关，可以通过 `#pragma pack` 修改。

4.2 计算 `PerInfo` 的大小

假设在 64 位平台，默认对齐：

- `char` 对齐 1 字节
- `int` 对齐 4 字节
- `double` 对齐 8 字节
- 最大对齐为 8

结构体定义：

```
typedef struct PerInfo {
    char name[10];
    char gender;
    int age;
    double height;
} PerInfo;
```

成员偏移计算：

成员	类型	大小	对齐要求	起始偏移	结束偏移	说明
name	char[10]	10	1	0	9	从 0 开始，占 10 字节
gender	char	1	1	10	10	紧接在 name 后面
age	int	4	4	12	15	偏移 11 不是 4 的倍数，填充 1 字节到偏移 12
height	double	8	8	16	23	偏移 16 是 8 的倍数，无需填充

总大小目前为 24 字节。
最大对齐要求是 8，24 是 8 的整数倍，因此不需要在结构体末尾额外填充。

所以：

```
sizeof(PerInfo) = 24 字节
```

验证代码：

```
#include <stdio.h>
```

```
typedef struct PerInfo {
    char name[10];
    char gender;
    int age;
    double height;
} PerInfo;

int main() {
    printf("sizeof(PerInfo) = %zu\n", sizeof(PerInfo));
    return 0;
}
```

在常见 64 位系统上输出：

```
sizeof(PerInfo) = 24
```

4.3 注意

- 如果编译器设置了不同的对齐方式，例如 `#pragma pack(1)`，结构体大小可能变为 $10 + 1 + 4 + 8 = 23$ 字节。
- 如果平台或编译器默认对齐不同，结果也可能变化。
- 一般情况下，上述 `PerInfo` 在 64 位默认对齐下为 **24 字节**。

C语言链表问题解答

1. 什么是链表

链表是一种线性数据结构，它由一组节点组成。每个节点通常包含两部分：

- **数据域**：用于存储元素本身。
- **指针域**：用于存储下一个节点的地址。

链表中的节点在内存中不要求连续存放，而是通过指针把各个节点按逻辑顺序连接起来。通常使用一个“头指针”指向链表的第一个节点。如果头指针为 `NULL`，表示链表为空。最后一个节点的指针域通常为 `NULL`，表示链表结束。

2. 链表和数组的区别

链表和数组都是线性结构，但它们在存储方式、访问方式、插入删除效率等方面有明显区别。

比较项	数组	链表
存储方式	元素在内存中连续存放	节点在内存中离散存放，通过指针连接

比较项	数组	链表
大小	通常固定，动态数组扩容时需要重新分配并复制	可以动态增减，按需分配节点
随机访问	支持下标访问，时间复杂度 $O(1)$	不支持随机访问，访问第 k 个元素需要从头遍历，时间复杂度 $O(n)$
插入和删除	平均需要移动大量元素，时间复杂度 $O(n)$	如果已经找到位置，只需修改指针，时间复杂度 $O(1)$ ；但查找位置仍需 $O(n)$
内存开销	只存储数据本身，可能预留多余空间	每个节点额外存储一个或多个指针，内存开销更大
缓存友好性	连续内存，缓存命中率高	节点分散，缓存命中率通常较低
适用场景	频繁随机访问、元素数量相对稳定	频繁插入删除、元素数量变化较大

```
#include <stdio.h> #include <stdlib.h> #include <stdbool.h>

typedef struct Node { int data; struct Node *next; // 原为 struct node *next; 大小写错误 } Node;

Node* creatnode(int data) { Node* newnode = (Node*)malloc(sizeof(Node)); if (newnode == NULL) { return NULL; } newnode->data = data; newnode->next = NULL; return newnode; }

Node* createhead() { Node* newnode = creatnode(0); return newnode; }

void addfirst(Node *head, int data) { Node newnode = creatnode(data); newnode->next = (*head)->next; (*head)->next = newnode; }

void addtail(Node head, int data) { Node newnode = creatnode(data); Node cur = *head; while (cur->next != NULL) { cur = cur->next; } cur->next = newnode; }

void add(Node head, int data, int pos) { if (pos == 0) { addfirst(head, data); } else { Node newnode = creatnode(data); Node cur = *head; int i = 0; while (i < pos) { cur = cur->next; if (cur == NULL) { return; } i++; } newnode->next = cur->next; cur->next = newnode; } }

bool findnode(Node *head, int pos) { Node cur = *head; int i = 0; while (i < pos) { cur = cur->next; if (cur == NULL) { return false; } i++; } printf("pos=%d,data=%d", pos, cur->data); return true; }

bool delfirst(Node *head) { Node cur = *head; if (cur->next == NULL) { return false; } cur = cur->next; (*head)->next = cur->next; cur->next = NULL; free(cur); return true; }

bool deltail(Node *head) { Node cur = head; if (cur->next == NULL) { return false; } while ((cur->next)->next != NULL) { cur = cur->next; } Node target = cur->next; cur->next = NULL; free(target); return true; }

bool del(Node *head, int pos) { Node cur = head; if (cur->next == NULL) { return false; } int i = 0; while (i < pos - 1) { cur = cur->next; if (cur == NULL) { return false; } i++; } Node target = cur->next; cur->next = target->next; target->next = NULL; free(target); return true; }
```

```
bool changenode(Node *head, int pos, int data) { Node cur = *head; if (cur->next == NULL) { return false; } int i = 0; while (i < pos) { cur = cur->next; if (cur == NULL) { return false; } i++; } cur->data = data; return true; }

void turnaround(Node **head) { if ((head)->next == NULL) { return; } Node first = (head)->next; (head)->next = NULL; Node cur = first; Node turn = head; while (first->next != NULL) { while ((cur->next)->next != NULL) { cur = cur->next; } Node target = cur->next; cur->next = NULL; turn->next = target; turn = turn->next; cur = first; } turn->next = first; }
```